

Timéo BERNARD GOARANT

Lycée Vauban, Brest

Gatien JAUMOUILLE

Nolan RIBETTE

BTS CIEL IR



Drone cible d'entraînement pour la marine nationale



X



Sebastien GUIVARCH

Denis CAILLE

Francesa BREHONNET

1. Introduction	3
1.1 Contexte du projet.....	3
1.2 Objectif du projet.....	3
1.3 Intérêt du projet.....	4
2. Présentation des outils du projet.....	4
2.1 GitHub.....	4
2.2 Google Drive.....	5
2.3 Synthèse.....	5
3. Environnement de développement.....	6
3.1 Justification des outils et langages choisis.....	6
3.2 Justification des librairies utilisées.....	6
3.3 Identification des contraintes du client.....	6
4. Architecture du projet.....	7
4.1 Diagramme d'architecture logicielle et matérielle.....	7
4.2 Schéma réseau.....	9
5. Développement – état d'avancement.....	10
5.1 Composants développés.....	10
5.2 Extraits de code.....	11
5.2.1 Initialisation du nœud et souscription au topic.....	11
5.2.2 Traitement des données de la manette.....	12
5.2.3 Construction de la trame et envoi.....	12
5.3 Tableau de tests.....	13
6. Exploitation réseau et matériels.....	13
6.1 Présentation des équipements.....	13
6.2 Justification des choix matériels.....	13
6.3 Résultat d'analyse réseau avec wireshark.....	14
6.4 Gestion des versions.....	14
6.5 Cybersécurité.....	14

1.Introduction

1.1 Contexte du projet

Ce projet s'inscrit dans le cadre du BTS CIEL option A au lycée Vauban de Brest, en partenariat avec la Marine nationale et l'IRENAV (Institut de Recherche de l'École Navale).

Il répond à un besoin opérationnel réel de la Marine : entraîner les équipages à faire face à des menaces navales modernes.

Depuis plusieurs années, notamment en mer Rouge, des attaques sont menées contre des navires militaires ou civils à l'aide de petits navires rapides (skiffs) ou de drones de surface autonomes (aussi appelé USV : Unmanned Surface Vehicle). Ces menaces sont peu coûteuses, rapides, difficiles à détecter et représentent un risque important.

Dans ce contexte, la Marine nationale souhaite disposer de drones cibles réalistes, capables de simuler ce type d'attaque lors d'exercices. Le projet qui nous est confié consiste donc à concevoir un prototype de drone naval de surface, à faible coût, destiné à être utilisé lors d'entraînements.

1.2 Objectif du projet

L'objectif principal du projet est de concevoir et réaliser un prototype fonctionnel de drone naval de type USV servant de cible d'entraînement pour la Marine nationale.

Plus précisément, le drone doit être capable de :

- Être piloté à distance par un opérateur (mode manuel),
- Suivre un itinéraire autonome à partir de points GPS (mode autonome),
- Se verrouiller sur une cible détectée par caméra et se diriger vers elle (mode verrouillage),
- Fonctionner avec des technologies peu coûteuses et des composants du commerce (approche DIY),
- Être suffisamment fiable pour des essais réels tout en restant économique, car il est conçu pour être potentiellement détruit lors des exercices.

Ce projet sert également de preuve de concept, afin d'évaluer la faisabilité de solutions low-cost pour des applications d'entraînement militaire.

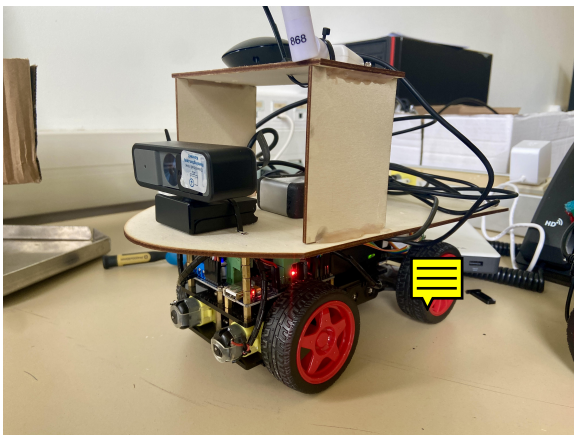


Photo 1 : Photo du prototype du drone cible

1.3 Intérêt du projet

Ce projet présente un triple intérêt : opérationnel, technologique et pédagogique.

- Intérêt opérationnel
 - Il répond à un besoin concret de la Marine nationale en matière d'entraînement face à des menaces modernes.
 - Il permet de simuler des scénarios réalistes sans mobiliser des équipements coûteux.
 - Il contribue à améliorer la préparation et la réactivité des équipages.
- Intérêt technologique
 - Le projet mobilise des technologies actuelles : ROS 2, GPS, vision par caméra, communication LoRa, Docker.
 - Il explore l'utilisation de solutions low-cost dans un contexte militaire, ce qui est stratégique.
- Intérêt pédagogique
 - Il nous permet, étudiants, de travailler sur un projet réel et professionnalisant.
 - Il développe des compétences clés du BTS CIEL : conception système, développement logiciel, communication, tests, documentation et travail en équipe.
 - Il met les étudiants en situation proche de l'ingénierie industrielle et de la recherche appliquée.

2. Présentation des outils du projet

2.1 GitHub

GitHub est utilisé comme outil central de gestion du projet. Il permet de centraliser l'ensemble du code source, d'assurer le travail collaboratif entre les membres de l'équipe et de conserver l'historique des modifications ainsi qu'un suivi du projet. Grâce au système de versions, chaque évolution du projet est tracée et sécurisée, ce qui facilite la correction des erreurs et la validation des développements. GitHub est également utilisé pour héberger la documentation technique, garantissant ainsi la continuité, la fiabilité et la maintenabilité du projet.

Dans le GitHub, nous avons créé une arborescence simple et intuitive.

- SysML : contient les différents SysML du projet.
- Simulation : contient les scripts de tests et de simulation du projet.
- Docs technique : l'entièreté des documentation technique des membres du groupe.
- Code : vous y trouverez le code source du projet.

Les fonctionnalités de gestion de projet de GitHub ont été utilisées pour organiser et suivre l'avancement du travail. Les issues permettent de décrire les tâches à réaliser, les anomalies à corriger ou les améliorations à apporter, tout en les attribuant à un membre de l'équipe. Le tableau Kanban offre une vision claire de l'état du projet en classant les tâches selon leur progression (à faire, en cours, terminé). Le travail est découpé en sprints, correspondant à des périodes de développement définies, durant lesquelles des objectifs précis sont fixés. Cette organisation permet un suivi rigoureux, une meilleure répartition des tâches et une adaptation rapide en cas de difficulté.

2.2 Google Drive

Google Drive est utilisé comme espace de stockage et de partage des documents du projet. Il permet de centraliser les fichiers non liés au code, tels que les documents de conception, les comptes rendus de réunion, et les schémas. Cet outil facilite le travail collaboratif en assurant un accès commun aux informations à jour, tout en évitant les doublons et les pertes de données. Google Drive contribue ainsi à une organisation efficace et à la continuité du projet.

Voici son arborescence :

- Compte rendu : contient les comptes rendu des différents sprints.
- Schémas : les différents schémas en lien avec notre projet.
- Soutenance : contient des documents utiles pour la rédaction de la soutenance, le rapport commun ainsi que 3 dossiers :
 - Timéo BERNARD GOARANT
 - Gatien JAUMOUILLE
 - Nolan RIBETTE

Ils contiennent les rédactions de soutenance de chaque membres du groupe.

Google Drive nous sert bien uniquement à déposer des documentations ainsi que des schéma. Vous n'y trouverez en aucun cas du codes dedans.

2.3 Synthèse

Cette organisation a été choisie afin de séparer clairement le développement logiciel de la gestion documentaire. GitHub est dédié au code source, au suivi des tâches et à la collaboration technique, tandis que Google Drive est utilisé pour le partage et la centralisation des documents du projet. Cette complémentarité facilite le travail en équipe en offrant à chacun un accès rapide aux informations nécessaires, réduit les risques d'erreurs ou de doublons et améliore la coordination entre les membres du groupe. Elle permet ainsi un suivi structuré, efficace et collaboratif du projet.

3. Environnement de développement

3.1 Justification des outils et langages choisis

Pour ce projet, nous avons choisi d'utiliser ROS2 comme framework principal de communication entre les différents composants robotisés. ROS2 offre une architecture distribuée, adaptée à l'intégration de multiples capteurs et actionneurs, et permet une gestion efficace des messages entre modules via des topics et services.

L'application tourne sur une Raspberry Pi 5, qui fournit une puissance de calcul suffisante pour gérer la capture vidéo, le traitement des données de capteurs et la communication LoRa en temps réel. L'utilisation de Docker permet d'isoler les environnements et de garantir une portabilité entre différentes plateformes. En parallèle, une VM Ubuntu 22.04 est utilisée avec ROS2 installé nativement, afin de faciliter le développement et le test des modules dans un environnement plus stable et contrôlé.

Le choix du Python comme langage principal s'explique par sa simplicité, sa lisibilité et la richesse de ses bibliothèques, particulièrement adaptées au traitement des capteurs ainsi qu'à la communication ROS2. Python permet également une intégration rapide avec les librairies ROS2, facilitant le prototypage et les tests fonctionnels.

3.2 Justification des librairies utilisées

Plusieurs librairies Python et packages ROS2 sont utilisés dans le projet :

- `rclpy` et `rclpy.node` : fournissent les fonctionnalités de base pour créer des nœuds ROS2, publier et souscrire à des topics, et gérer les services.
- `sensor_msgs.msg import Joy` : utilisé pour la gestion des commandes depuis un joystick ou tout autre interface de contrôle.
- `geometry_msgs.msg import Twist` : Permet de gérer les commandes de déplacement du robot ou des modules mobiles.
- `serial` : Librairie standard Python utilisée pour la communication avec des modules matériels via des ports série (ex. IMU, GPS).

Ces librairies ont été choisies pour leur compatibilité directe avec ROS2 et la Raspberry Pi, ainsi que pour leur robustesse et leur documentation complète, ce qui assure un développement fiable et modulable.

3.3 Identification des contraintes du client

Plusieurs contraintes techniques ont été définies pour répondre aux besoins du projet.

La première contrainte concerne la portée de communication. Le système doit permettre une communication stable sur une distance minimale de 3 kilomètres. Pour répondre à cette exigence, une communication radio utilisant la technologie LoRa en liaison point à point a été mise en place, grâce à deux antennes dédiées.

Une seconde contrainte concerne l'ergonomie de pilotage pour l'opérateur. Le système doit être facilement contrôlable, avec une interface intuitive. Pour cela, le pilotage est réalisé à l'aide d'une manette de jeu de type DualShock 4, offrant une prise en main familière et précise pour l'utilisateur.

Une autre contrainte importante concerne la qualité du flux vidéo. Le système doit permettre la diffusion d'un flux vidéo en temps réel provenant d'une webcam embarquée, avec une qualité suffisante pour permettre à l'opérateur de visualiser correctement l'environnement.

Enfin, certaines contraintes logicielles ont été imposées :

- L'ensemble du développement logiciel doit être réalisé avec ROS2 afin d'assurer une architecture modulaire et distribuée.
- L'interface utilisateur doit être développée avec Node-RED, qui permet de créer rapidement des interfaces graphiques interactives et de visualiser les données du système.

L'environnement de développement mis en place permet de répondre aux besoins techniques du projet tout en garantissant une architecture flexible et évolutive. L'utilisation de Robot Operating System 2 (ROS2) associée au langage Python facilite la communication entre les différents modules du système et l'intégration des capteurs. Le déploiement sur Raspberry Pi 5, ainsi que l'utilisation de conteneurs Docker et d'un environnement de développement sous Ubuntu 22.04, permet également d'assurer une bonne portabilité et une gestion maîtrisée de l'environnement logiciel.

Les outils et technologies sélectionnés permettent ainsi de répondre aux contraintes du projet, notamment en matière de communication longue portée, d'ergonomie de pilotage et de diffusion du flux vidéo.

Afin de mieux comprendre l'organisation globale du système, la partie suivante présentera l'architecture du projet. Celle-ci décrira la structure logicielle et matérielle du système à travers différents diagrammes, ainsi que l'organisation du réseau et la circulation des données entre les différents composants.

4. Architecture du projet

L'architecture d'un système permet de représenter l'organisation des différents composants matériels et logiciels ainsi que leurs interactions. Dans ce projet, le système s'appuie sur Robot Operating System 2 (ROS2) déployé sur une Raspberry Pi 5, permettant la communication entre les capteurs, les modules de transmission LoRa et l'interface opérateur développée avec Node-RED.

Afin de mieux comprendre l'organisation globale du projet, cette partie présentera l'architecture logicielle et matérielle du système ainsi qu'un schéma réseau.

4.1 Diagramme d'architecture logicielle et matérielle

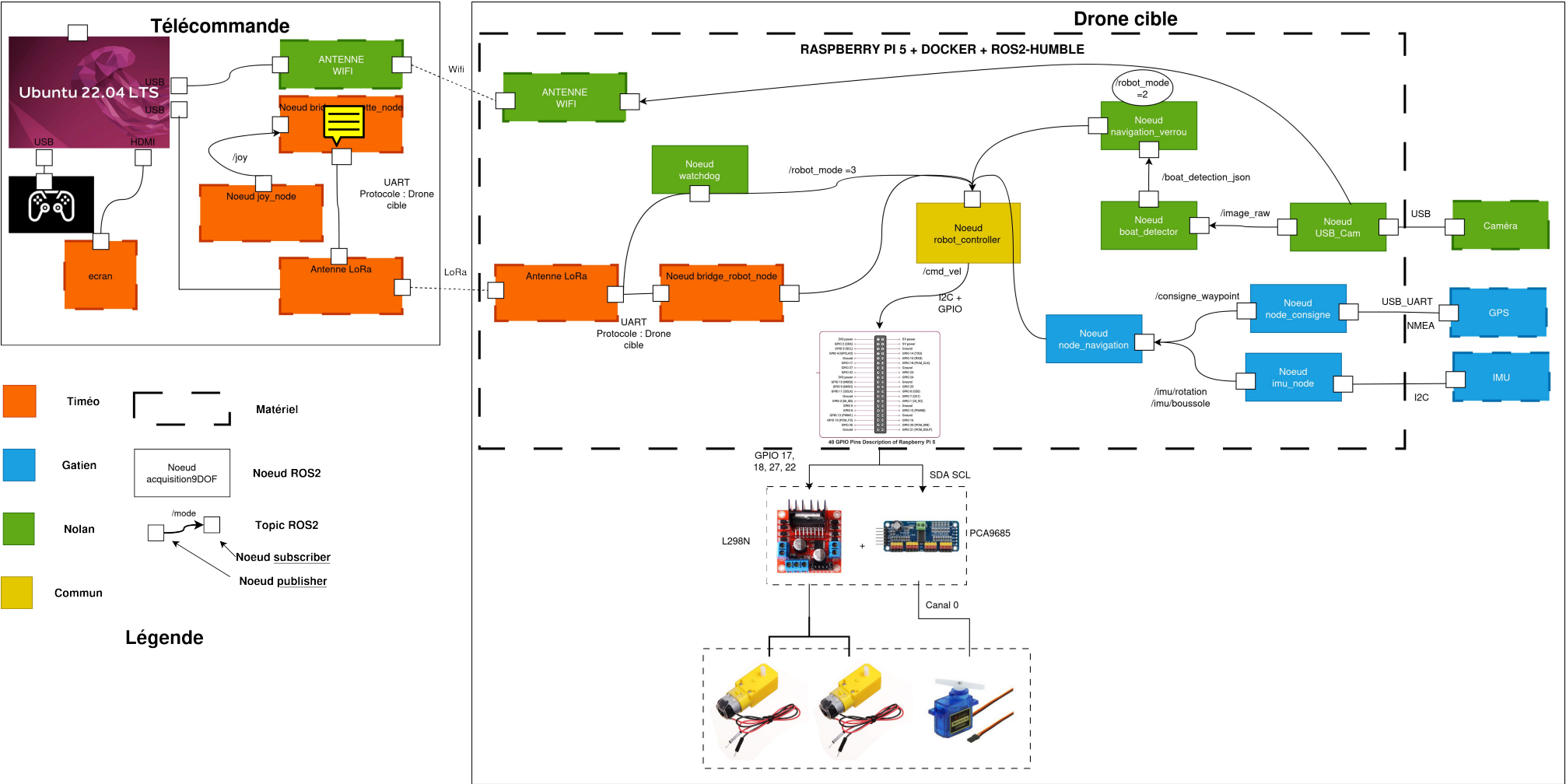


Schéma 2 : Diagramme d'architecture globale matérielle et logicielle du système du Drone cible

Ce schéma présente l'architecture complète du système en combinant les aspects matériels et logiciels. Il met en évidence l'organisation du poste opérateur (télécommande), la communication radio ainsi que le système embarqué sur le drone cible. L'ensemble du système repose sur ROS2 exécuter sur la Raspberry Pi 5 avec des conteneurs Docker, permettant de structurer et étanchéifier l'application sous forme de nœuds interconnectés via les topics ROS2. Les différents capteurs du drone (caméra, GPS et IMU) alimentent les modules de navigation et de détection, tandis que les commandes envoyées par l'opérateur sont transmises via une communication radio longue portée utilisant la technologie LoRa. Cette architecture permet de séparer clairement les fonctions de contrôle, de perception et de navigation du système.

La partie télécommande correspond au poste opérateur et regroupe les éléments permettant de piloter le drone à distance. Elle repose sur un ordinateur exécutant Ubuntu 22.04, connecté à une manette de jeu utilisée pour générer les commandes de pilotage. Le nœud `joy_node` récupère les informations provenant de la manette et les publie sous forme de messages ROS2. Ces données sont ensuite traitées par le nœud `bridge_manette_node`, qui assure l'adaptation et la transmission des commandes vers le système embarqué. La communication avec le drone est réalisée grâce à deux antennes LoRa en point-à-point (P2P), permettant d'assurer une liaison radio longue portée (supérieur à 5km en zone dégagée). L'ensemble de ces nœuds constitue l'interface de contrôle entre l'opérateur et le drone, en convertissant les actions de l'utilisateur en commandes exploitables par le système embarqué.

4.2 Schéma réseau

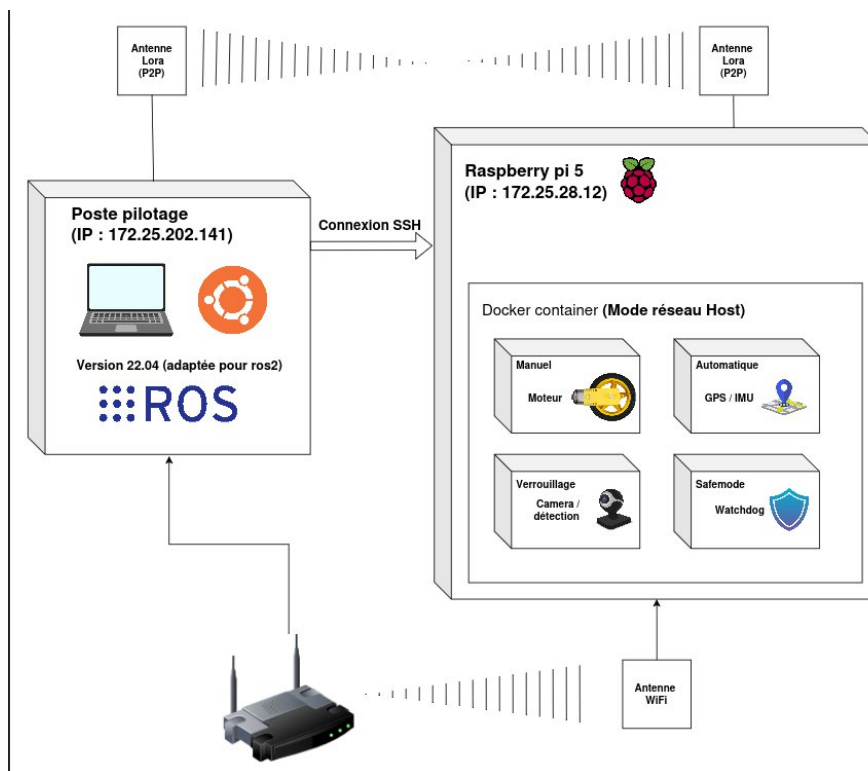


Schéma 3 : Schéma architecture réseau en développement

Ce schéma présente l'architecture réseau utilisée pour assurer la communication entre le poste de pilotage et le système embarqué sur la Raspberry Pi 5. Le poste opérateur fonctionne sous Ubuntu 22.04 avec ROS2 installé nativement, permettant le contrôle et la supervision du système. La communication principale entre les deux équipements est réalisée via une liaison radio longue portée LoRa en mode point à point, tandis qu'une connexion Wi-Fi permet l'accès réseau pour la configuration et la maintenance via SSH. Sur la Raspberry Pi, les différents modules fonctionnels sont exécutés dans un conteneur Docker configuré en mode réseau hôte, permettant aux différents services (pilotage manuel, navigation automatique basée sur GPS/IMU, détection par caméra et mécanismes de sécurité) de communiquer efficacement avec le système ROS2 et les interfaces réseau. Cette architecture assure une séparation claire entre le poste de contrôle et le système embarqué tout en garantissant une communication fiable entre les deux.

5. Développement – état d'avancement

5.1 Composants développés

Dans le cadre de ce projet, plusieurs composants logiciels ont été développés afin d'assurer la communication entre le poste de pilotage et le robot, ainsi que l'affichage des informations pour l'opérateur. Ces composants reposent sur l'architecture distribuée de ROS2.

– bridge_manette_node :

Le bridge_manette_node est exécuté sur la machine virtuelle fonctionnant sous Ubuntu 22.04. Ce nœud a pour rôle de récupérer les données issues de la manette de contrôle et de les transmettre au robot en utilisant un protocole de communication spécifiquement développé dans le cadre du projet.

Le nœud s'abonne au topic `/joy`. Les données sont ensuite encapsulées dans une trame respectant le protocole que nous avons défini pour la communication radio. Cette trame est envoyée au robot via le module de communication LoRa, ce qui permet d'assurer le pilotage du robot à longue distance.

Ce nœud assure également la réception des trames provenant du robot. Les informations contenues dans ces trames (position GPS, données IMU ou informations issues de la détection d'objets) sont décodées puis transmises vers l'interface utilisateur afin d'être affichées.

– bridge_robot_node :

Le bridge_robot_node est exécuté sur la Raspberry Pi 5 embarquée sur le robot. Son rôle principal est de servir d'interface entre la communication radio et l'architecture ROS2 du robot.

Ce nœud reçoit les trames de direction du drone envoyées par le poste opérateur et les décode afin d'extraire les informations de commande. Ces données sont ensuite converties en messages ROS2 et publiées sur le topic `/cmd_vel`, permettant ainsi de contrôler les déplacements du robot.

En parallèle, ce nœud collecte plusieurs informations provenant des différents capteurs embarqués, notamment :

- Les données de position du GPS,
- Les données d'orientation issues de l'IMU,

- Les informations provenant du système de détection basé sur la caméra (position et dimension du rectangle de détection généré par l'algorithme de vision Yolo V5).

Ces données sont ensuite encapsulées dans une trame du protocole de communication et transmises vers le poste opérateur afin d'être exploitées par le dashboard Node-Red.

– Interface Node-red :

L'interface utilisateur a été développée à l'aide de Node-Red. Cette interface permet à l'opérateur de visualiser en temps réel les informations transmises par le robot.

Elle affiche notamment :

- La position GPS du robot,
- Les informations issues de l'IMU,
- Les données liées à la détection de bateaux,
- Les informations nécessaires au pilotage du drone.

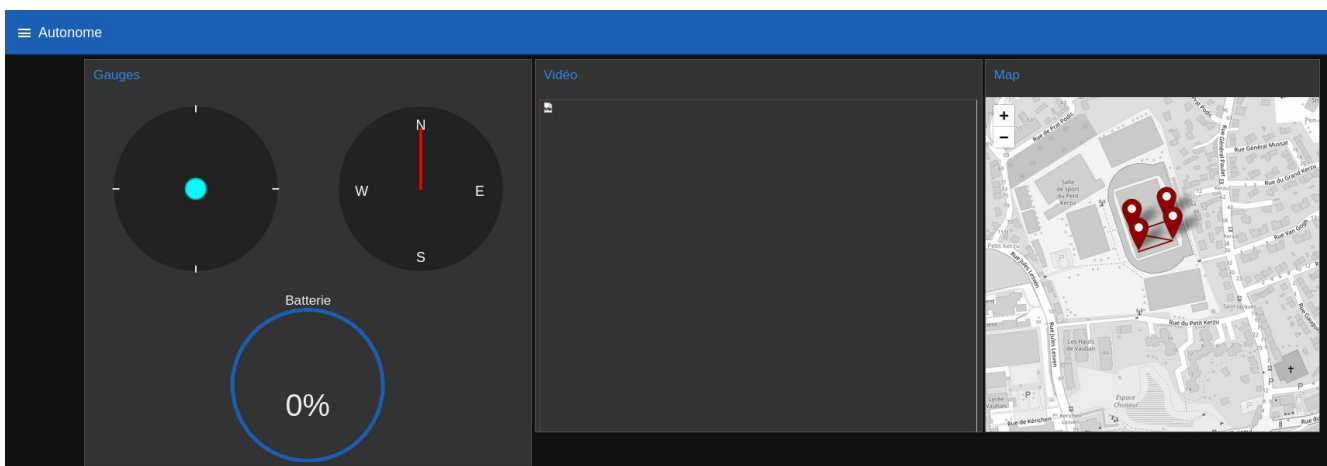


Photo 1 : Capture d'écran de l'interface Node-Red

L'utilisation de Node-RED permet de créer rapidement une interface graphique interactive et facilement modifiable, facilitant ainsi la supervision et le contrôle du robot pendant son utilisation.

5.2 Extraits de code

L'extrait de code suivant correspond au nœud `bridge_manette_node.py`, développé en Python pour l'environnement ROS2. Ce nœud est exécuté sur la machine virtuelle Ubuntu 22.04 et permet de transmettre les commandes de la manette vers le robot la communication LoRa.

5.2.1 Initialisation du nœud et souscription au topic

```
class Bridge_manette(Node):
    def __init__(self):
        super().__init__('bridge_manette')
        self.subscription = self.create_subscription(
```

```
joy,  
    '/joy',  
    self.listener_callback,  
    10)  
  
try:  
    self.ser = serial.Serial('/dev/ttyUSB0', 9600, timeout=0.1)  
    self.get_logger().info("UART ouvert avec succès")  
except serial.SerialException as e:  
    self.get_logger().error(f"Erreur ouverture UART: {e}")  
  
self.ser = None
```

Dans cette première partie, la classe `Bridge_manette` est définie comme un nœud ROS2. Le nœud souscrit au topic `/joy`, qui contient les données provenant de la manette. À chaque réception d'un message, la fonction `listener_callback` est appelée pour traiter les données.

Le code initialise également une communication série (UART) avec le module LoRa grâce à la bibliothèque `serial`. Cette liaison permet d'envoyer les commandes vers le robot.

5.2.2 Traitement des données de la manette

```
def listener_callback(self, msg):  
    x0, x1 = msg.axes[:2]  
    x0_mapped = int((x0+1)*127.5)  
    x1_mapped = int(x1*10+10)
```

Cette fonction est exécutée à chaque réception d'un message provenant de la manette. Les valeurs des deux axes du joystick de gauche de la manette sont récupérées dans `msg.axes`.

Ces valeurs étant initialement comprises entre -1 et 1, elles sont transformées en valeurs entières adaptées au protocole de communication utilisé pour piloter le robot. La valeur `x0` est alors comprise entre 0 et 255 avec un offset de 127.5 et la valeur `x1` est elle comprise entre 0 et 20 avec un offset de 10.

5.2.3 Construction de la trame et envoi

```
frame = bytearray([  
    0x52,      # Start  
    0x08,      # Type  
    0x02,      # Length  
    x0_mapped, # Payload[0]  
    x1_mapped  # Payload[1]  
)
```

```
self.get_logger().info(f"Trame: {[hex(b) for b in frame]}")

if self.ser and self.ser.is_open:

    # conversion en hex

    payload_hex = frame.hex()

    # construction commande AT format envoi pour le LoRa

    cmd = f"AT+SEND=0,{payload_hex},0,0\r\n"

    self.get_logger().info(f"Commande LoRa: {cmd.strip()}")

    # envoi

    self.ser.write(cmd.encode())
```

Dans cette partie du code, une trame de communication personnalisée est construite sous forme de tableau d'octets. Cette trame contient un octet de début, un type de message, la longueur du message et les valeurs correspondant aux commandes du joystick.

La trame est ensuite convertie en format hexadécimal afin d'être intégrée dans une commande AT, utilisée pour communiquer avec le module LoRa via le port série. La commande est finalement envoyée au module radio afin de transmettre les commandes au robot.

5.3 Tableau de tests

A COMPLETER QUAND LES TABLEAU SERONT FAIT

6. Exploitation réseau et matériels

6.1 Présentation des équipements



Dans le cadre de ce projet, plusieurs équipements matériels sont utilisés afin d'assurer le pilotage manuel du robot et la communication entre le poste opérateur et le système embarqué.

La Raspberry Pi 5 constitue l'ordinateur embarqué du robot. Elle exécute les différents nœuds de ROS2 nécessaires au fonctionnement du système.

La communication longue portée entre le poste opérateur et le robot est assurée par une antenne LoRa équipé d'un adaptateur USB LA66. La technologie LoRa permet d'établir une liaison radio fiable sur plusieurs kilomètres tout en consommant peu d'énergie.

Le pilotage du robot est réalisé à l'aide d'une DualShock 3 ou 4. Les informations provenant des joysticks sont récupérées sous forme de messages ROS2 afin de générer les commandes de déplacement du robot.

Enfin, le poste opérateur est constitué d'un ordinateur exécutant une machine virtuelle, grâce à Oracle VirtualBox, sous Ubuntu 22.04. A terme, la distribution Ubuntu 22.04 pourra être installer directement sur un PC. Cet environnement permet d'exécuter les nœuds ROS2 nécessaires au contrôle du système ainsi que l'interface développée avec Node-RED.

6.2 Justification des choix matériels

Le choix des équipements utilisés dans ce projet repose sur plusieurs critères, notamment les performances, la simplicité d'intégration et l'adaptation aux besoins du système.

La communication entre le poste opérateur et le robot repose sur la technologie LoRa. Ce choix s'explique principalement par sa grande portée de communication en champ libre, permettant d'atteindre plusieurs kilomètres tout en restant fiable. De plus, cette technologie permet d'envoyer des données personnalisées, ce qui facilite l'intégration de notre protocole adapté aux besoins du projet.

Le système embarqué utilise une Raspberry Pi 5. Cette carte a été choisie pour sa capacité de calcul élevée, sa mémoire vive suffisante et la possibilité d'exécuter une distribution Linux, ce qui permet de faire fonctionner facilement ROS2 et les différents services nécessaires au projet.

Le pilotage du robot est réalisé à l'aide d'une DualShock 3. Cette manette offre une ergonomie adaptée à une utilisation prolongée, ce qui est important pour l'opérateur lors des phases de pilotage. Les manettes de jeu étant conçues pour les joueurs qui les utilisent pendant de longues périodes, elles offrent une prise en main confortable et des commandes précises.

Enfin, l'interface utilisateur a été développée avec Node-RED. Plusieurs outils comme RViz et rqt ont été testés au début du projet, mais Node-RED s'est révélé plus adapté aux besoins du projet. Il permet de créer rapidement une interface graphique complète et facilement modifiable. De plus, cet outil ayant déjà été utilisé lors de travaux pratiques précédents, sa prise en main a été rapide, ce qui a facilité le développement de l'interface.

6.3 Résultat d'analyse réseau avec wireshark

6.4 Gestion des versions

La gestion du code source du projet est assurée à l'aide de l'outil Git, avec un dépôt hébergé sur la plateforme GitHub.

Le projet est organisé au  d'une branche principale unique (main), sur laquelle l'ensemble du développement est centralisé. Le code est mis à jour régulièrement grâce à des commits fréquents, permettant de suivre l'évolution du projet et de conserver un historique des modifications.

6.5 Cybersécurité

Plusieurs mesures ont été mises en place afin d'améliorer la sécurité du système. L'accès à la Raspberry Pi 5 se fait à distance via le protocole SSH, en utilisant pour les développeur une clé SSH, ce qui permet d'éviter l'utilisation de mots de passe pour l'authentification et est donc plus rapide. La Raspberry Pi ainsi que la machine virtuelle sous Ubuntu 22.04 sont également protégées par des mots de passe afin de limiter les accès non autorisés.

Le réseau Wi-Fi utilisé pour les communications locales est sécurisé avec le protocole WPA2-Personal, garantissant un chiffrement des échanges sur le réseau sans fil.

La communication radio entre les deux modules utilisant la technologie LoRa est configurée en mode point à point privé, ce qui limite les possibilités d'interception ou d'accès par des équipements extérieurs.

Enfin, l'utilisation de Docker sur la Raspberry Pi permet d'isoler les différents programmes selon le mode de fonctionnement du robot. Cette isolation des services améliore la sécurité globale du système en limitant les interactions directes entre les différentes applications.